Review II – Screenshot Checklist

Work through this list from top to bottom in one SQL*Plus session (Run SQL Command Line). The order is the execution order, which is slightly different from the order of the figures in the report.

Before you start

- Start from an empty schema. If you ran the scripts before, run `@99_cleanup.sql` first (ignore "does not exist" errors).
- Put all files of the sql/ folder in one folder and start SQL*Plus from that folder, or use the full path, e.g. `@C:\dbms\sql\01_ddl.sql`.
- Make the console window wide (about 150 columns) so rows do not wrap: Properties → Layout → Window size width 150.
- Run `@00_setup_display.sql` at the start of every new session (after reconnecting as well).
- For each figure: clear the screen (`CLEAR SCREEN`), paste the block from the file, press Enter, then take the screenshot (Windows: Win + Shift + S). Include the typed command and the full output; crop away the empty part of the window.
- Save as screenshots/fig_NN.png using the figure number below (e.g. fig_14.png). Then either paste each picture into the placeholder box in the DOCX, or run `python3 tools/build_report.py` to insert them automatically.
- If any output differs from "What you should see", do not edit the screenshot. Note the difference and fix the script or ask for help.

Step 1: @00_setup_display.sql then @01_ddl.sql

1. Figure 2 – Running 01_ddl.sql: four sequences and eight tables created

Save as **screenshots/fig_02.png**

```
@00_setup_display.sql
@01_ddl.sql
```

What you should see: "Sequence created." 4 times, then "Table created." 8 times. No ORA- errors.

Step 2: @02_insert_data.sql (no screenshot; the last lines must be "3 rows updated." / "Commit complete.")

Step 3: 03_check_schema_data.sql – run each block separately

2. Figure 3 – Structure of the BOOKING and PAYMENT tables (DESC)

Save as **screenshots/fig_03.png** | Source: 03_check_schema_data.sql, block SC_DESC

```
DESC booking
DESC payment
```

What you should see: BOOKING: 13 columns, 8 shown as NOT NULL (7 + the primary key). PAYMENT: 7 columns, 5 NOT NULL.

3. Figure 4 – Number of constraints of each kind, read from USER_CONSTRAINTS

Save as **screenshots/fig_04.png** | Source: 03_check_schema_data.sql, block SC_COUNT

```
SELECT constraint_kind, COUNT(*) AS total
FROM  (SELECT CASE constraint_type
                WHEN 'P' THEN 'PRIMARY KEY'
                WHEN 'R' THEN 'FOREIGN KEY'
                WHEN 'U' THEN 'UNIQUE'
                WHEN 'C' THEN CASE WHEN constraint_name LIKE 'SYS%' THEN 'NOT NULL' ELSE 'CHECK' END
              END AS constraint_kind
       FROM   user_constraints
       WHERE  table_name IN ('BRANCH', 'VEHICLE_CATEGORY', 'VEHICLE', 'CUSTOMER',
                             'EMPLOYEE', 'BOOKING', 'PAYMENT', 'MAINTENANCE'))
GROUP  BY constraint_kind
ORDER  BY total DESC;
```

What you should see: NOT NULL 34, CHECK 23, FOREIGN KEY 10, UNIQUE 10, PRIMARY KEY 8 (5 rows).

4. Figure 5 – The ten foreign keys and the table each one refers to

Save as **screenshots/fig_05.png** | Source: 03_check_schema_data.sql, block SC_FK

```
SELECT c.table_name, c.constraint_name, r.table_name AS refers_to, c.delete_rule
FROM   user_constraints c
JOIN   user_constraints r ON r.constraint_name = c.r_constraint_name
WHERE  c.constraint_type = 'R'
ORDER  BY c.table_name, c.constraint_name;
```

What you should see: 10 rows. FK_PAYMENT_BOOKING shows DELETE_RULE = CASCADE; all others NO ACTION.

5. Figure 8 – Row count of every table after loading 02_insert_data.sql

Save as **screenshots/fig_08.png** | Source: 03_check_schema_data.sql, block DATA_COUNTS

```
SELECT 'BRANCH' AS table_name, COUNT(*) AS total_rows FROM branch
UNION ALL SELECT 'VEHICLE_CATEGORY', COUNT(*) FROM vehicle_category
UNION ALL SELECT 'VEHICLE',          COUNT(*) FROM vehicle
UNION ALL SELECT 'CUSTOMER',         COUNT(*) FROM customer
UNION ALL SELECT 'EMPLOYEE',         COUNT(*) FROM employee
UNION ALL SELECT 'BOOKING',          COUNT(*) FROM booking
UNION ALL SELECT 'PAYMENT',          COUNT(*) FROM payment
UNION ALL SELECT 'MAINTENANCE',      COUNT(*) FROM maintenance;
```

What you should see: BRANCH 10, VEHICLE_CATEGORY 10, VEHICLE 14, CUSTOMER 12, EMPLOYEE 15, BOOKING 22, PAYMENT 45, MAINTENANCE 12.

6. Figure 9 – VEHICLE table after loading: current odometer, status and branch of each vehicle

Save as **screenshots/fig_09.png** | Source: 03_check_schema_data.sql, block DATA_VEHICLE

```
SELECT vehicle_id, reg_number, brand, model, fuel_type, odometer, status, category_id, branch_id
FROM   vehicle
ORDER  BY vehicle_id;
```

What you should see: 14 rows. Vehicles 2 and 5 RENTED, 8 MAINTENANCE, 14 RETIRED, the rest AVAILABLE.

7. Figure 10 – BOOKING table after loading: 16 completed, 2 ongoing, 2 booked and 2 cancelled rentals

Save as **screenshots/fig_10.png** | Source: 03_check_schema_data.sql, block DATA_BOOKING

```
SELECT booking_id, customer_id, vehicle_id, pickup_branch_id, return_branch_id,
       start_date, end_date, status
FROM   booking
ORDER  BY booking_id;
```

What you should see: 22 rows. Bookings 1-16 COMPLETED, 17-18 ONGOING, 19-20 BOOKED, 21-22 CANCELLED.

Step 4: 04_constraint_tests.sql – two blocks, every statement must fail

8. Figure 6 – Constraint tests 1-4: UNIQUE, CHECK and NOT NULL violations rejected

Save as **screenshots/fig_06.png** | Source: 04_constraint_tests.sql, block CT1

```
-- 1. UNIQUE: this registration number already belongs to vehicle 1
INSERT INTO vehicle (vehicle_id, reg_number, brand, model, fuel_type, category_id, branch_id)
VALUES (50, 'TN09AB1001', 'Maruti Suzuki', 'Baleno', 'PETROL', 1, 1);
-- 2. CHECK: HYBRID is not one of the allowed fuel types
INSERT INTO vehicle (vehicle_id, reg_number, brand, model, fuel_type, category_id, branch_id)
VALUES (51, 'TN09ZZ9999', 'Toyota', 'Hyryder', 'HYBRID', 3, 1);
-- 3. CHECK: branch phone must have exactly 10 digits
INSERT INTO branch VALUES (50, 'DriveEase Test', 'Chennai', 'Test address', '98401', NULL);
-- 4. NOT NULL: licence number is missing
INSERT INTO customer (customer_id, name, phone, license_expiry, dob)
VALUES (50, 'Test User', '9000000050', DATE '2035-01-01', DATE '1995-01-01');
```

What you should see: ORA-00001 (UQ_VEHICLE_REG), ORA-02290 (CK_VEHICLE_FUEL), ORA-02290 (CK_BRANCH_PHONE), ORA-01400 (CUSTOMER.LICENSE_NO).

9. Figure 7 – Constraint tests 5-8: foreign key and table-level CHECK violations rejected

Save as **screenshots/fig_07.png** | Source: 04_constraint_tests.sql, block CT2

```
-- 5. FOREIGN KEY (child side): booking 999 does not exist
INSERT INTO payment VALUES (500, 999, 1000, SYSDATE, 'CASH', 'ADVANCE', 'SUCCESS');
-- 6. FOREIGN KEY (parent side): customer 1 still has bookings
```

```
DELETE FROM customer WHERE customer_id = 1;
-- 7. Table-level CHECK: end date before start date
UPDATE booking SET end_date = start_date - 1 WHERE booking_id = 1;
-- 8. Table-level CHECK (BR-08): end odometer lower than start odometer
UPDATE booking SET end_odometer = 100 WHERE booking_id = 1;
ROLLBACK;
```

What you should see: ORA-02291 (FK_PAYMENT_BOOKING), ORA-02292 (FK_BOOKING_CUSTOMER), ORA-02290 (CK_BOOKING_DATES), ORA-02290 (CK_BOOKING_ODOMETER), then "Rollback complete."

Step 5: 05_queries.sql – run Q1 to Q16 one at a time

10. Figure 14 – Q1 output: vehicles free in Chennai from 21-Oct to 23-Oct

Save as **screenshots/fig_14.png** | Source: 05_queries.sql, block Q1

```
SELECT v.vehicle_id, v.reg_number, v.brand, v.model, vc.category_name, vc.daily_rate, b.branch_name
FROM   vehicle v
JOIN   vehicle_category vc ON vc.category_id = v.category_id
JOIN   branch b            ON b.branch_id    = v.branch_id
WHERE  b.city   = 'Chennai'
AND    v.status = 'AVAILABLE'
AND    NOT EXISTS (SELECT 1 FROM booking bk
                   WHERE  bk.vehicle_id = v.vehicle_id
                   AND    bk.status IN ('BOOKED', 'ONGOING')
                   AND    bk.start_date < TO_DATE('2026-10-23 10:00', 'YYYY-MM-DD HH24:MI')
                   AND    bk.end_date   > TO_DATE('2026-10-21 10:00', 'YYYY-MM-DD HH24:MI'))
ORDER  BY vc.daily_rate;
```

What you should see: 3 rows: vehicle 12 (Activa, 500), 4 (City, 2200), 3 (Innova Crysta, 3200). Vehicle 1 is not listed.

11. Figure 15 – Q2 output: fleet status of every branch

Save as **screenshots/fig_15.png** | Source: 05_queries.sql, block Q2

```
SELECT b.branch_name,
       COUNT(v.vehicle_id)                                        AS fleet,
       SUM(CASE WHEN v.status = 'AVAILABLE'   THEN 1 ELSE 0 END) AS available,
       SUM(CASE WHEN v.status = 'RENTED'      THEN 1 ELSE 0 END) AS rented,
       SUM(CASE WHEN v.status = 'MAINTENANCE' THEN 1 ELSE 0 END) AS in_maint,
       SUM(CASE WHEN v.status = 'RETIRED'     THEN 1 ELSE 0 END) AS retired
FROM   branch b
LEFT JOIN vehicle v ON v.branch_id = b.branch_id
GROUP  BY b.branch_id, b.branch_name
ORDER  BY b.branch_id;
```

What you should see: 10 rows. Hyderabad Hitech City has FLEET 0; Coimbatore has IN_MAINT 1; Chennai Airport has RETIRED 1.

12. Figure 16 – Q3 output: net revenue collected by each pickup branch

Save as **screenshots/fig_16.png** | Source: 05_queries.sql, block Q3

```
SELECT br.branch_name,
       COUNT(DISTINCT bk.booking_id) AS bookings,
       SUM(CASE WHEN p.payment_type = 'REFUND' THEN -p.amount ELSE p.amount END) AS net_revenue
FROM   payment p
JOIN   booking bk ON bk.booking_id = p.booking_id
JOIN   branch  br ON br.branch_id  = bk.pickup_branch_id
WHERE  p.txn_status = 'SUCCESS'
GROUP  BY br.branch_name
ORDER  BY net_revenue DESC;
```

What you should see: 10 rows, highest Kochi Edappally 16500, lowest Secunderabad 1500.

13. Figure 17 – Q4 output: month-wise collection split by payment type

Save as **screenshots/fig_17.png** | Source: 05_queries.sql, block Q4

```
SELECT TO_CHAR(payment_date, 'YYYY-MM') AS month,
       SUM(CASE WHEN payment_type = 'ADVANCE'  THEN amount ELSE 0 END) AS advance,
       SUM(CASE WHEN payment_type = 'FINAL'    THEN amount ELSE 0 END) AS final,
       SUM(CASE WHEN payment_type = 'LATE_FEE' THEN amount ELSE 0 END) AS late_fee,
       SUM(CASE WHEN payment_type = 'REFUND'   THEN amount ELSE 0 END) AS refund
FROM   payment
WHERE  txn_status = 'SUCCESS'
GROUP  BY TO_CHAR(payment_date, 'YYYY-MM')
ORDER  BY month;
```

What you should see: 4 rows (2026-07 to 2026-10). 2026-10 shows REFUND 2500.

14. Figure 18 – Q5 output: top three customers by amount paid

Save as **screenshots/fig_18.png** | Source: 05_queries.sql, block Q5

```
SELECT *
FROM  (SELECT c.customer_id, c.name, COUNT(DISTINCT bk.booking_id) AS bookings,
              SUM(p.amount) AS total_paid
       FROM   customer c
       JOIN   booking bk ON bk.customer_id = c.customer_id
       JOIN   payment p  ON p.booking_id   = bk.booking_id
       WHERE  p.txn_status = 'SUCCESS' AND p.payment_type <> 'REFUND'
       GROUP  BY c.customer_id, c.name
       ORDER  BY total_paid DESC)
WHERE  ROWNUM <= 3;
```

What you should see: Anjali Thomas 17500, Vikram Reddy 14900, Harish Kumar 13700.

15. Figure 19 – Q6 output: customers who paid more than the average customer

Save as **screenshots/fig_19.png** | Source: 05_queries.sql, block Q6

```
SELECT c.customer_id, c.name, SUM(p.amount) AS total_paid
FROM   customer c
JOIN   booking bk ON bk.customer_id = c.customer_id
JOIN   payment p  ON p.booking_id   = bk.booking_id
WHERE  p.txn_status = 'SUCCESS' AND p.payment_type <> 'REFUND'
GROUP  BY c.customer_id, c.name
HAVING SUM(p.amount) > (SELECT AVG(t.spent)
                        FROM  (SELECT SUM(p2.amount) AS spent
                               FROM   payment p2
                               JOIN   booking b2 ON b2.booking_id = p2.booking_id
                               WHERE  p2.txn_status = 'SUCCESS' AND p2.payment_type <> 'REFUND'
                               GROUP  BY b2.customer_id) t)
ORDER  BY total_paid DESC;
```

What you should see: 6 rows, from Anjali Thomas 17500 down to Meera Krishnan 11000.

16. Figure 20 – Q7 output: registered customers who have never made a booking

Save as **screenshots/fig_20.png** | Source: 05_queries.sql, block Q7

```
SELECT customer_id, name, phone FROM customer
MINUS
SELECT c.customer_id, c.name, c.phone
FROM   customer c JOIN booking bk ON bk.customer_id = c.customer_id;
```

What you should see: 1 row: customer 12, Karthik Sundar.

17. Figure 21 – Q8 output: rentals, average days and average distance per category

Save as **screenshots/fig_21.png** | Source: 05_queries.sql, block Q8

```
SELECT vc.category_name,
       COUNT(DISTINCT v.vehicle_id)                    AS vehicles,
       COUNT(bk.booking_id)                            AS rentals,
       ROUND(AVG(bk.end_date - bk.start_date), 1)      AS avg_days,
       ROUND(AVG(bk.end_odometer - bk.start_odometer)) AS avg_km
FROM   vehicle_category vc
LEFT JOIN vehicle v  ON v.category_id = vc.category_id
LEFT JOIN booking bk ON bk.vehicle_id = v.vehicle_id AND bk.status = 'COMPLETED'
GROUP  BY vc.category_name
ORDER  BY rentals DESC, vc.category_name;
```

What you should see: 10 rows. Hatchback, SUV and Sedan have 3 rentals each; MUV has AVG_KM 780.

18. Figure 22 – Q9 output: one-way rentals with pickup and return branch names

Save as **screenshots/fig_22.png** | Source: 05_queries.sql, block Q9

```
SELECT bk.booking_id, c.name AS customer, v.reg_number,
       pb.branch_name AS pickup_branch, rb.branch_name AS return_branch, bk.status
FROM   booking bk
JOIN   customer c ON c.customer_id = bk.customer_id
JOIN   vehicle  v ON v.vehicle_id  = bk.vehicle_id
JOIN   branch  pb ON pb.branch_id  = bk.pickup_branch_id
JOIN   branch  rb ON rb.branch_id  = bk.return_branch_id
WHERE  bk.pickup_branch_id <> bk.return_branch_id
ORDER  BY bk.booking_id;
```

What you should see: 3 rows: bookings 2, 15 and 16.

19. Figure 23 – Q10 output: late returns, hours late and late fee billed

Save as **screenshots/fig_23.png** | Source: 05_queries.sql, block Q10

```
SELECT bk.booking_id, c.name AS customer, bk.end_date, bk.actual_return_date,
       CEIL(ROUND((bk.actual_return_date - bk.end_date) * 1440) / 60) AS hours_late,
       vc.late_fee_per_hr,
       NVL(SUM(p.amount), 0) AS fee_billed
FROM   booking bk
JOIN   customer c          ON c.customer_id  = bk.customer_id
JOIN   vehicle v           ON v.vehicle_id   = bk.vehicle_id
JOIN   vehicle_category vc ON vc.category_id = v.category_id
LEFT JOIN payment p        ON p.booking_id   = bk.booking_id AND p.payment_type = 'LATE_FEE'
WHERE  bk.actual_return_date > bk.end_date
GROUP  BY bk.booking_id, c.name, bk.end_date, bk.actual_return_date, vc.late_fee_per_hr
ORDER  BY bk.booking_id;
```

What you should see: 5 rows: bookings 2 (4 h, 1000), 4 (1 h, 250), 7 (3 h, 900), 12 (3 h, 900), 16 (2 h, 600).

20. Figure 24 – Q11 output: payments that are pending or failed

Save as **screenshots/fig_24.png** | Source: 05_queries.sql, block Q11

```
SELECT p.payment_id, p.booking_id, c.name AS customer, v.reg_number,
       p.payment_type, p.amount, p.txn_status
FROM   payment p
JOIN   booking  bk ON bk.booking_id = p.booking_id
JOIN   customer c  ON c.customer_id = bk.customer_id
JOIN   vehicle  v  ON v.vehicle_id  = bk.vehicle_id
WHERE  p.txn_status IN ('PENDING', 'FAILED')
ORDER  BY p.payment_id;
```

What you should see: 3 rows: payments 30 (PENDING), 38 (FAILED), 45 (PENDING).

21. Figure 25 – Q12 output: cancelled bookings with advance and refund amounts

Save as **screenshots/fig_25.png** | Source: 05_queries.sql, block Q12

```
SELECT bk.booking_id, c.name AS customer, bk.start_date,
       MAX(CASE WHEN p.payment_type = 'REFUND' THEN p.payment_date END) AS cancelled_on,
       SUM(CASE WHEN p.payment_type = 'ADVANCE' THEN p.amount ELSE 0 END) AS advance,
       SUM(CASE WHEN p.payment_type = 'REFUND'  THEN p.amount ELSE 0 END) AS refund
FROM   booking bk
JOIN   customer c ON c.customer_id = bk.customer_id
LEFT JOIN payment p ON p.booking_id = bk.booking_id
WHERE  bk.status = 'CANCELLED'
GROUP  BY bk.booking_id, c.name, bk.start_date
ORDER  BY bk.booking_id;
```

What you should see: 2 rows: booking 21 advance 2000 refund 2000; booking 22 advance 1000 refund 500.

22. Figure 26 – Q13 output: vehicles whose total maintenance cost is above Rs 5000

Save as **screenshots/fig_26.png** | Source: 05_queries.sql, block Q13

```
SELECT v.reg_number, v.brand || ' ' || v.model AS vehicle,
       COUNT(m.maint_id)  AS jobs,
       NVL(SUM(m.cost), 0) AS total_cost
FROM   vehicle v
LEFT JOIN maintenance m ON m.vehicle_id = v.vehicle_id
GROUP  BY v.reg_number, v.brand, v.model
HAVING NVL(SUM(m.cost), 0) > 5000
ORDER  BY total_cost DESC;
```

What you should see: 8 rows, starting with MH12ST9010 (E-Class) 42000.

23. Figure 27 – Q14 output: vehicles that are currently out of service and why

Save as **screenshots/fig_27.png** | Source: 05_queries.sql, block Q14

```
SELECT v.reg_number, v.brand || ' ' || v.model AS vehicle, 'RENTED' AS state,
       'With ' || c.name || ', due ' || TO_CHAR(bk.end_date, 'DD-Mon HH24:MI') AS details
FROM   vehicle v
JOIN   booking  bk ON bk.vehicle_id = v.vehicle_id AND bk.status = 'ONGOING'
JOIN   customer c  ON c.customer_id = bk.customer_id
UNION ALL
SELECT v.reg_number, v.brand || ' ' || v.model, 'MAINTENANCE',
       m.service_type || ': ' || m.description
FROM   vehicle v
JOIN   maintenance m ON m.vehicle_id = v.vehicle_id AND m.maint_status = 'IN_PROGRESS';
```

What you should see: 3 rows: Nexon EV and Creta RENTED, Dzire MAINTENANCE.

24. Figure 28 – Q15 output: agents who handled two or more bookings

Save as **screenshots/fig_28.png** | Source: 05_queries.sql, block Q15

```
SELECT e.emp_id, e.name AS employee, br.branch_name, COUNT(bk.booking_id) AS bookings_handled
FROM   employee e
JOIN   branch br ON br.branch_id = e.branch_id
LEFT JOIN booking bk ON bk.emp_id = e.emp_id
WHERE  e.role = 'AGENT'
GROUP  BY e.emp_id, e.name, br.branch_name
HAVING COUNT(bk.booking_id) >= 2
ORDER  BY bookings_handled DESC, e.emp_id;
```

What you should see: 7 rows, Divya Menon 5 and Pooja Rao 4 at the top.

25. Figure 29 – Q16 output: customers whose licence expires in the next 12 months

Save as **screenshots/fig_29.png** | Source: 05_queries.sql, block Q16

```
SELECT customer_id, name, phone, license_no, license_expiry,
       TRUNC(license_expiry - SYSDATE) AS days_left
FROM   customer
WHERE  license_expiry BETWEEN SYSDATE AND ADD_MONTHS(SYSDATE, 12)
ORDER  BY license_expiry;
```

What you should see: 2 rows: Sneha Pillai (30-APR-27) and Divya Nair (31-AUG-27). DAYS_LEFT depends on the run date.

Step 6: 06_dml_demo.sql – three blocks

26. Figure 11 – INSERT: new vehicle 15 added; omitted columns take their DEFAULT values

Save as **screenshots/fig_11.png** | Source: 06_dml_demo.sql, block DML_INSERT

```
-- A new Kia Seltos joins the Chennai Airport branch. transmission, odometer and status
-- are not given, so the DEFAULT values ('MANUAL', 0, 'AVAILABLE') are stored.
INSERT INTO vehicle (vehicle_id, reg_number, brand, model, mfg_year, fuel_type, category_id, branch_id)
VALUES (15, 'TN09YZ2015', 'Kia', 'Seltos', 2024, 'PETROL', 3, 2);
COMMIT;
SELECT vehicle_id, reg_number, brand, model, transmission, odometer, status, branch_id
FROM   vehicle WHERE vehicle_id = 15;
```

What you should see: "1 row created." Row shows TRANSMISSION MANUAL, ODOMETER 0, STATUS AVAILABLE, BRANCH_ID 2.

27. Figure 12 – UPDATE: salary increment for two agents and change of a customer's contact details

Save as **screenshots/fig_12.png** | Source: 06_dml_demo.sql, block DML_UPDATE

```
-- 10% increment for the agents of branches 1 and 3 (the busiest branches in Q15)
SELECT emp_id, name, branch_id, salary FROM employee WHERE role = 'AGENT' AND branch_id IN (1, 3);
UPDATE employee SET salary = salary * 1.10 WHERE role = 'AGENT' AND branch_id IN (1, 3);
SELECT emp_id, name, branch_id, salary FROM employee WHERE role = 'AGENT' AND branch_id IN (1, 3);
-- Customer 7 changed phone number and address
UPDATE customer SET phone = '9841999107', address = 'Chromepet, Chennai' WHERE customer_id = 7;
SELECT customer_id, name, phone, address FROM customer WHERE customer_id = 7;
COMMIT;
```

What you should see: Salaries 32000 -> 35200 and 31000 -> 34100; "2 rows updated."; customer 7 phone 9841999107.

28. Figure 13 – DELETE: cancelled booking 22 removed; its two payments are deleted by ON DELETE CASCADE

Save as **screenshots/fig_13.png** | Source: 06_dml_demo.sql, block DML_DELETE

```
-- Booking 22 was cancelled. Deleting it also deletes its payments (ON DELETE CASCADE).
SELECT payment_id, booking_id, payment_type, amount FROM payment WHERE booking_id = 22;
DELETE FROM booking WHERE booking_id = 22 AND status = 'CANCELLED';
SELECT COUNT(*) AS payments_left FROM payment WHERE booking_id = 22;
COMMIT;
```

What you should see: Payments 41 (ADVANCE 1000) and 43 (REFUND 500) listed, "1 row deleted.", then PAYMENTS_LEFT = 0.

Step 7: @07_functions.sql, @08_procedures.sql, @09_cursors.sql, @10_triggers.sql (no screenshot; each object must say "created.")

Step 8: 11_transactions.sql – run the blocks strictly in this order (IDs come from sequences)

29. Figure 30 – All functions, procedures and triggers compiled with status VALID

Save as **screenshots/fig_30.png** | Source: 11_transactions.sql, block T_OBJECTS

```
SELECT object_name, object_type, status
FROM   user_objects
WHERE  object_type IN ('FUNCTION', 'PROCEDURE', 'TRIGGER')
ORDER  BY object_type, object_name;
```

What you should see: 17 rows (6 FUNCTION, 7 PROCEDURE, 4 TRIGGER), every STATUS = VALID.

30. Figure 31 – fn_rental_days, fn_calculate_rent, fn_late_fee and fn_booking_balance used in a query

Save as **screenshots/fig_31.png** | Source: 11_transactions.sql, block T_FUNCS1

```
SELECT booking_id, end_date, actual_return_date,
       fn_rental_days(start_date, end_date) AS days,
       fn_calculate_rent(booking_id)        AS rent,
       fn_late_fee(booking_id)              AS late_fee,
       fn_booking_balance(booking_id)       AS balance
FROM   booking
WHERE  booking_id IN (1, 2, 4, 7, 12, 16)
ORDER  BY booking_id;
```

What you should see: Booking 2: DAYS 3, RENT 8400, LATE_FEE 1000, BALANCE 0. Booking 12: BALANCE 900. Booking 16: BALANCE 4000.

31. Figure 32 – fn_age for three customers and fn_is_vehicle_free for two vehicles

Save as **screenshots/fig_32.png** | Source: 11_transactions.sql, block T_FUNCS2

```
SELECT customer_id, name, dob, fn_age(dob, SYSDATE) AS age_today
FROM   customer WHERE customer_id IN (4, 8, 9);
SELECT fn_is_vehicle_free(1,  TO_DATE('2026-10-21 10:00', 'YYYY-MM-DD HH24:MI'),
                              TO_DATE('2026-10-23 10:00', 'YYYY-MM-DD HH24:MI')) AS vehicle1_free,
       fn_is_vehicle_free(12, TO_DATE('2026-10-21 10:00', 'YYYY-MM-DD HH24:MI'),
                              TO_DATE('2026-10-23 10:00', 'YYYY-MM-DD HH24:MI')) AS vehicle12_free
FROM   dual;
```

What you should see: Ages 25, 24, 47 (in Oct 2026). VEHICLE1_FREE = 0 (booking 19 overlaps), VEHICLE12_FREE = 1.

32. Figure 33 – sp_register_customer: customer 13 registered

Save as **screenshots/fig_33.png** | Source: 11_transactions.sql, block T_REGISTER

```
EXEC sp_register_customer('Rohit Verma', '9841000113', 'rohit.verma@gmail.com', 'TN0920200045678', DATE '2032-
06-30', DATE '1998-09-12', 'Velachery, Chennai');
SELECT customer_id, name, phone, license_no, registered_on FROM customer WHERE customer_id = 13;
```

What you should see: "Customer registered. ID = 13" and the new row for Rohit Verma.

33. Figure 34 – sp_create_booking: booking 23 created with a Rs 400 advance

Save as **screenshots/fig_34.png** | Source: 11_transactions.sql, block T_BOOK

```
-- Customer 13 books the scooter (vehicle 12) at Chennai Central for 2 days from tomorrow 10:00,
-- returning it at Chennai Airport (one-way), and pays Rs 400 advance by UPI.
EXEC sp_create_booking(13, 12, 2, 1, 2, TRUNC(SYSDATE) + 1 + 10/24, TRUNC(SYSDATE) + 3 + 10/24, 400, 'UPI');
SELECT booking_id, customer_id, vehicle_id, start_date, end_date, status FROM booking WHERE booking_id = 23;
SELECT payment_id, booking_id, amount, method, payment_type FROM payment WHERE booking_id = 23;
```

What you should see: "Booking 23 created for vehicle 12. Estimated rent: Rs 1000, advance paid: Rs 400"; status BOOKED; payment 46 ADVANCE 400 UPI.

34. Figure 35 – sp_pickup_vehicle: booking 23 becomes ONGOING and the vehicle becomes RENTED

Save as **screenshots/fig_35.png** | Source: 11_transactions.sql, block T_PICKUP

```
EXEC sp_pickup_vehicle(23);
SELECT b.booking_id, b.status, b.start_odometer, v.vehicle_id, v.status AS vehicle_status
FROM   booking b JOIN vehicle v ON v.vehicle_id = b.vehicle_id WHERE b.booking_id = 23;
```

What you should see: "Booking 23 is now ONGOING. Start odometer: 9060"; VEHICLE_STATUS = RENTED.

35. Figure 36 – sp_return_vehicle: rent, final payment and late fee for a late one-way return

Save as **screenshots/fig_36.png** | Source: 11_transactions.sql, block T_RETURN

```
-- Returned at Chennai Airport after driving 60 km, 3 hours after the end time.
EXEC sp_return_vehicle(23, 9120, TRUNC(SYSDATE) + 3 + 13/24, 'CASH');
SELECT booking_id, status, actual_return_date, start_odometer, end_odometer FROM booking WHERE booking_id = 23;
SELECT vehicle_id, status, branch_id, odometer FROM vehicle WHERE vehicle_id = 12;
SELECT payment_id, payment_type, amount, method FROM payment WHERE booking_id = 23 ORDER BY payment_id;
```

What you should see: "Rent: Rs 1000, advance: Rs 400, final payment: Rs 600, late fee: Rs 150"; vehicle 12 AVAILABLE, BRANCH_ID 2, ODOMETER 9120; payments 46 ADVANCE 400, 47 FINAL 600, 48 LATE_FEE 150.

36. Figure 37 – sp_cancel_booking: full refund for early cancellation, 50% for late cancellation

Save as **screenshots/fig_37.png** | Source: 11_transactions.sql, block T_CANCEL

```
-- Booking 24 starts in 10 days (full refund); booking 25 starts in 5 hours (50% refund).
EXEC sp_create_booking(13, 13, 5, 3, 3, TRUNC(SYSDATE) + 10 + 9/24, TRUNC(SYSDATE) + 11 + 9/24, 400, 'CARD');
EXEC sp_cancel_booking(24);
EXEC sp_create_booking(13, 13, 5, 3, 3, SYSDATE + 5/24, SYSDATE + 1 + 5/24, 400, 'CARD');
EXEC sp_cancel_booking(25);
SELECT booking_id, payment_type, amount, method FROM payment WHERE booking_id IN (24, 25) ORDER BY payment_id;
```

What you should see: Booking 24: refund Rs 400. Booking 25: refund Rs 200. Payments 49-52 listed.

37. Figure 38 – Explicit cursor (sp_overdue_report): rentals past their end time

Save as **screenshots/fig_38.png** | Source: 11_transactions.sql, block T_OVERDUE

```
EXEC sp_overdue_report;
```

What you should see: Booking 17 (KA03GH4005, Kavya Iyer) and, after 15-Oct 10:00, booking 18. Hours and fee depend on the run time.

38. Figure 39 – Cursor FOR loop (sp_pending_dues): completed bookings with money still to collect

Save as **screenshots/fig_39.png** | Source: 11_transactions.sql, block T_DUES

```
EXEC sp_pending_dues;
```

What you should see: Booking 12 Farhan Ali Rs 900, booking 16 Sneha Pillai Rs 4000, total Rs 4900.

39. Figure 40 – trg_customer_validate rejects an under-age customer and an expired licence

Save as **screenshots/fig_40.png** | Source: 11_transactions.sql, block T_TRG_CUSTOMER

```
-- Both registrations must be rejected by trg_customer_validate
EXEC sp_register_customer('Arav Shah', '9000000001', NULL, 'TN0920250000001', DATE '2035-01-01', DATE '2010-01-
01');
EXEC sp_register_customer('Nikhil Das', '9000000002', NULL, 'TN0920000000002', DATE '2025-01-01', DATE '1990-
01-01');
```

What you should see: ORA-20201 "Customer must be at least 21 years old. Age: 16" and ORA-20202 "Driving licence has expired on 01-JAN-2025".

40. Figure 41 – trg_booking_validate rejects a short licence and a vehicle that is already rented

Save as **screenshots/fig_41.png** | Source: 11_transactions.sql, block T_TRG_BOOKING

```
-- Customer 4's licence expires on 30-APR-2027, before this rental ends
EXEC sp_create_booking(4, 1, 2, 1, 1, TO_DATE('2027-05-01 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2027-05-03
10:00', 'YYYY-MM-DD HH24:MI'), 1000, 'UPI');
-- Vehicle 5 is RENTED (booking 17 is still ongoing)
EXEC sp_create_booking(1, 5, 6, 4, 4, TRUNC(SYSDATE) + 20 + 9/24, TRUNC(SYSDATE) + 22 + 9/24, 0);
```

What you should see: ORA-20212 "Driving licence expires on 30-APR-2027, before the rental ends" and ORA-20213 "Vehicle 5 cannot be booked. Status: RENTED".

41. Figure 42 – trg_maintenance_sync: vehicle 8 released, vehicle 3 sent to the workshop, two blocked actions

Save as **screenshots/fig_42.png** | Source: 11_transactions.sql, block T_TRG_MAINT

```
SELECT vehicle_id, reg_number, status FROM vehicle WHERE vehicle_id IN (3, 8);
-- Clutch job on vehicle 8 is finished; a new tyre job starts on vehicle 3
UPDATE maintenance SET maint_status = 'DONE', cost = 9800 WHERE maint_id = 11;
INSERT INTO maintenance (maint_id, vehicle_id, service_type, description, maint_status)
VALUES (seq_maintenance.NEXTVAL, 3, 'TYRE', 'Wheel alignment and tyre rotation', 'IN_PROGRESS');
COMMIT;
SELECT vehicle_id, reg_number, status FROM vehicle WHERE vehicle_id IN (3, 8);
-- Vehicle 3 is now in the workshop, so it cannot be booked
EXEC sp_create_booking(13, 3, 3, 2, 2, TRUNC(SYSDATE) + 5 + 10/24, TRUNC(SYSDATE) + 6 + 10/24, 0);
-- Vehicle 2 is out on rent, so maintenance cannot start
INSERT INTO maintenance (maint_id, vehicle_id, service_type, description, maint_status)
VALUES (seq_maintenance.NEXTVAL, 2, 'GENERAL', 'Periodic service', 'IN_PROGRESS');
```

What you should see: Before: 3 AVAILABLE, 8 MAINTENANCE. After: 3 MAINTENANCE, 8 AVAILABLE. Then ORA-20213 (Status: MAINTENANCE) and ORA-20221.

42. Figure 43 – Business-rule violations rejected by the procedures

Save as **screenshots/fig_43.png** | Source: 11_transactions.sql, block T_RULES

```
-- 1. BR-02: vehicle 1 already has booking 19 from 20-Oct to 22-Oct
EXEC sp_create_booking(10, 1, 2, 1, 1, TO_DATE('2026-10-21 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-10-23
10:00', 'YYYY-MM-DD HH24:MI'), 500, 'UPI');
-- 2. Vehicle 10 is at the Pune branch, not at Chennai Central
EXEC sp_create_booking(10, 10, 2, 1, 1, TO_DATE('2026-11-02 10:00', 'YYYY-MM-DD HH24:MI'), TO_DATE('2026-11-03
10:00', 'YYYY-MM-DD HH24:MI'), 0);
-- 3. Booking 23 is already COMPLETED, so it cannot be picked up again
EXEC sp_pickup_vehicle(23);
-- 4. Booking 18 is ONGOING, so it cannot be cancelled
EXEC sp_cancel_booking(18);
-- 5. Phone number already belongs to customer 1
EXEC sp_register_customer('Aditya R', '9841000101', NULL, 'TN0920259999999', DATE '2036-01-01', DATE '1996-03-
14');
```

What you should see: ORA-20103 (already booked), ORA-20102 (wrong branch), ORA-20120 (booking 23 COMPLETED), ORA-20140 (status ONGOING), ORA-20150 (already registered).